

1.

Output:

[you, are, how]

2.

Output:

2

10

4

6

6

3

3.

Output:

[1, 1, 6, 6, 2, 2]

[9, 9, 15, 15, 4, 4, -3, -3, 42, 42]

[40, 40, 50, 50, 60, 60, 10, 10, 20, 20, 30, 30]

4.

Output:

[1, 3, 5] [2, 4, 6]

[-3, 15, 9, 71] [42, 4]

[3] [30, 20, 10, 60, 50, 40, 0]

5.

Output:

[-1, -3, -5]

[-42, -4, -71]

[-10, -60, -50]

```

public static void splitStack(Stack<Integer> s) {
    Queue<Integer> q = new LinkedList<>();

    while (!s.isEmpty()) { // while stack is not empty
        q.add(s.pop());
    }

    int size = q.size();
    for (int i = 0; i < size; i++) { // for loop to process queue
        int n = q.remove(); // remove and store in n
        if (n < 0) { // if n is negative
            s.push(n); // push to stack
        } else { // if n is positive
            q.add(n); // add back to queue
        }
    }

    while (!q.isEmpty()) {
        s.push(q.remove()); // add remaining elements to stack
    }
}

```

```

public class SplitStack {
    public static void main(String[] args) {
        Stack<Integer> stack = new Stack<>();
        stack.push(5);
        stack.push(-10);
        stack.push(15);
        stack.push(0);
        stack.push(-5);
        stack.push(100);

        splitStack(stack);
        System.out.println(stack);
    }
}

```

duely@MacBook-Air-Duy CS211 % /usr/bin/env /Library/Interp /var/folders/w3/0x9k0jl565gczs9x9ztngbw0000gn/T/cp...
 -5, -10, 100, 0, 15, 5]
 duely@MacBook-Air-Duy CS211 %

6.

```

public static Stack<Integer> copyStack(Stack<Integer> s) {
    Stack<Integer> copy = new Stack<>();
    Queue<Integer> q = new LinkedList<>();

    // move elements from s to q
    while (!s.isEmpty()) {
        q.add(s.pop());
    }

    // move elements back to s, restores original order
    while (!q.isEmpty()) {
        s.push(q.remove());
    }

    // move elements back to copy whilst building copy
    while (!s.isEmpty()) {
        int n = s.pop();
        q.add(n);
        copy.push(n);
    }

    // restore original stack
    while (!q.isEmpty()) {
        s.push(q.remove());
    }

    return copy;
}

```

```

public class CopyStack {
    public static void main(String[] args) {
        Stack<Integer> stack = new Stack<>();
        stack.push(1);
        stack.push(2);
        stack.push(3);
        stack.push(4);
        stack.push(5);

        Stack<Integer> copy = copyStack(stack);
        System.out.println(stack);
        System.out.println(copy);
    }
}

```

duely@MacBook-Air-Duy CS211 % /usr/bin/env /Library/Interp s/Home/bin/java -cp /var/folders/w3/0x9k0jl565gczs9x9ztngbw0000gn/T/cp... week05.lab.CopyStack
 [1, 2, 3, 4, 5]
 [1, 2, 3, 4, 5]
 duely@MacBook-Air-Duy CS211 %

7.

```

public static void collapse(Stack<Integer> s) {
    Queue<Integer> q = new LinkedList<>();

    while (!s.isEmpty()) {
        q.add(s.pop()); // add elements to queue
    }

    Integer top = null; // Integer allows null, i
    if (q.size() % 2 == 1) { // if there's one r
        top = q.remove(); // remove and store in
    }

    while (q.size() >= 2) { // while queue has a
        int a = q.remove();
        int b = q.remove();
        s.push(a + b); // push a + b to stack
    }

    while (!s.isEmpty()) { // reverse stack
        q.add(s.pop());
    }

    while (!q.isEmpty()) { // push elements back
        s.push(q.remove());
    }

    if (top != null) { // if top is not null
        s.push(top); // push top back to stack
    }
}

```

```

public class CollapseStack {
    public static void main(String[] args) {
        Stack<Integer> stack = new Stack<>();
        stack.push(-1);
        stack.push(-2);
        stack.push(-3);
        stack.push(0);
        stack.push(3);
        stack.push(2);
        stack.push(1);

        collapse(stack);
        System.out.println(stack);
    }
}

```

duely@MacBook-Air-Duy CS211 % /usr/bin/env /Library/Interp cp /var/folders/w3/0x9k0jl565gczs9x9ztngbw0000gn/T/cp... [-3, -3, 5, 1]
 duely@MacBook-Air-Duy CS211 %

8.

9.

```

public static boolean isPalindrome(Queue<Integer> q) {
    Stack<Integer> s = new Stack<>();

    int size = q.size();
    for (int i = 0; i < size; i++) { // add all element
        int n = q.remove();
        s.push(n); // push to stack, reverses order
        q.add(n); // add back to queue
    }

    boolean isPalindrome = true;
    for (int i = 0; i < size; i++) {
        int a = q.remove();
        int b = s.pop();
        if (a != b) { // compare front of queue and top
            isPalindrome = false; // loop doesn't end b
        }
        q.add(a); // add back to queue
    }

    return isPalindrome;
}

public static void main(String[] args) {
    Queue<Integer> queue = new LinkedList<>();
    queue.add(3);
    queue.add(8);
    queue.add(17);
    queue.add(9);
    queue.add(17);
    queue.add(8);
    queue.add(3);
    System.out.println(isPalindrome(queue));
    System.out.println(queue);

    queue.add(0);
    System.out.println(isPalindrome(queue));
    System.out.println(queue);
}

```

PROBLEMS 117 OUTPUT DEBUG CONSOLE TERMINAL TEST

```

duey@MacBook-Air-Duy CS211 % /usr/bin/env /Library/Intern
cp /var/folders/w3/0x9k0j1s65gczs9x9ztngbw0000gn/T/cp_243
true
[3, 8, 17, 9, 17, 8, 3]
false
[3, 8, 17, 9, 17, 8, 3, 0]

```

10.

```

public static void mirror(Stack<Integer> s) {
    if (s == null) {
        throw new IllegalArgumentException();
    }

    Queue<Integer> q = new LinkedList<>();

    while (!s.isEmpty()) { // add all elements
        q.add(s.pop());
    }

    while (!q.isEmpty()) { // add all elements
        s.push(q.remove());
    }

    while (!s.isEmpty()) { // add all elements
        q.add(s.pop());
    }

    int size = q.size();
    // iterate over queue and add elements to
    for (int i = 0; i < size; i++) {
        int n = q.remove();
        s.push(n);
        q.add(n);
    }

    while (!s.isEmpty()) { // add remaining el
        q.add(s.pop());
    }

    while (!q.isEmpty()) { // add all elements
        s.push(q.remove());
    }
}

public static void main(String[] args) {
    Stack<Integer> stack = new Stack<>();
    stack.push(10);
    stack.push(50);
    stack.push(19);
    stack.push(54);
    stack.push(30);
    stack.push(67);
    mirror(stack);
    System.out.println(stack);
}

```

PROBLEMS 117 OUTPUT DEBUG CONSOLE TERMINAL

```

duey@MacBook-Air-Duy CS211 % /usr/bin/env /Library/Int
cp /var/folders/w3/0x9k0j1s65gczs9x9ztngbw0000gn/T/c
[10, 50, 19, 54, 30, 67, 67, 30, 54, 19, 50, 10]

```

11.

```

public static void compressDuplicates(Stack<Integer> s) {
    Queue<Integer> q = new LinkedList<>();

    while (!s.isEmpty()) { // add all elements to queue
        q.add(s.pop());
    }

    while (!q.isEmpty()) { // add all elements back to sta
        s.push(q.remove());
    }

    while (!s.isEmpty()) { // add all elements to queue, n
        q.add(s.pop());
    }

    int current = q.remove(); // remove and store first el
    int count = 1; // count is 1 because first element is
    while (!q.isEmpty()) {
        int next = q.remove();
        if (current == next) { // if next element is same
            count++;
        }
        else {
            s.push(count); // push current count
            s.push(current); // push current element

            current = next; // update current to next elem
            count = 1; // reset count
        }
    }

    // push last count and current pair (since while loop
    s.push(count);
    s.push(current);
}

public static void main(String[] args) {
    Stack<Integer> stack = new Stack<>();
    stack.push(2);
    stack.push(2);
    stack.push(2);
    stack.push(2);
    stack.push(2);
    stack.push(2);
    stack.push(-5);
    stack.push(-5);
    stack.push(3);
    stack.push(3);
    stack.push(3);
    stack.push(3);
    stack.push(4);
    stack.push(4);
    stack.push(1);
    stack.push(0);
    stack.push(17);
    stack.push(17);

    compressDuplicates(stack);
    System.out.println(stack);
}

```

PROBLEMS 117 OUTPUT DEBUG CONSOLE TERMINAL

duey@MacBook-Air-Duy CS211 % /usr/bin/env /Library/...
cp /var/folders/w3/0x9k0j1s65gczs9x9ztngbw000gn/T/...
[5, 2, 2, -5, 4, 3, 2, 4, 1, 1, 1, 0, 2, 17]
duey@MacBook-Air-Duy CS211 %

12.

```

public static void maxToTop(Stack<Integer> s) {
    Queue<Integer> q = new LinkedList<>();

    int max = s.peek(); // initialize max
    while (!s.isEmpty()) {
        int n = s.pop();
        q.add(n);
        if (n > max) {
            max = n; // update max
        }
    }

    while (!q.isEmpty()) { // add all elements back to stack
        s.push(q.remove());
    }

    while (!s.isEmpty()) { // add all elements back to queue
        q.add(s.pop());
    }

    // add all elements back to stack except max
    while (!q.isEmpty()) {
        int n = q.remove();
        if (n != max) {
            s.push(n);
        }
    }

    s.push(max); // add max to top of stack
}

public class MaxToTop {
    Run | Debug | Windsurf: Refactor | Explain | Generate Javadoc
    public static void main(String[] args) {
        Stack<Integer> stack = new Stack<>();
        stack.push(27);
        stack.push(5);
        stack.push(42);
        stack.push(-11);
        stack.push(0);
        stack.push(19);
        maxToTop(stack);
        System.out.println(stack);
    }
}

```

PROBLEMS 117 OUTPUT DEBUG CONSOLE TERMINAL

duey@MacBook-Air-Duy CS211 % /usr/bin/env /Library/...
cp /var/folders/w3/0x9k0j1s65gczs9x9ztngbw000gn/T/...
[27, 5, -11, 0, 19, 42]
duey@MacBook-Air-Duy CS211 %